Exercices : Utilisation de bibliothèques, modularité

💻 Exercice 1 - Bibliothèque `math`

- Dans la console Python, importez le module `math`
- En utilisant la fonction `ceil` du module, remplissez le tableau suivant :

x	-1.5	-2	1.5	2
`ceil(x)`				

- En utilisant la fonction `floor` du module, remplissez le tableau suivant :

x	-1.5	-2	1.5	2
`floor(x)`				

- En utilisant la fonction `sqrt` du module, remplissez le tableau suivant :

x	4	9	16	25
`sqrt(x)`				

- Calculez `pow(2,3)` et `pow(3,2)`
- Que fait la fonction `pow` ?
- Déterminez la valeur de l'hypoténuse d'un triangle rectangle ayant pour dimensions des côtés adjacents 2 et 1 cm. (Utilisez le théorème de Pythagore)
- Saisissez dans l'interpéteur `hypot(2, 1)`. Que remarquez-vous ?

💻 Exercice 2 - Bibliothèque `random`

- Dans la console Python, importez le module `random`
- Completez le tableau suivant :

Commande	1er essai	2ième essai	3ième essai
`random()`			
`randint(0, 3)`			
`choice(['a', 'b', 'c', 'd', 'e'])`			
`sample(['a', 'b', 'c', 'd', 'e'], 2)`			

💻 Exercice 3 - Bibliothèque `turtle`

Le Module Turtle permet de dessiner en programmant. Découvrons quelques possibilités offertes par ce module.

- Après avoir créé un nouveau fichier, saisissez le code suivant et exécutez le programme :

```Python
import turtle as t

def carré (pixel):
    for i in range(4):
        t.forward (pixel)
        t.left(90)

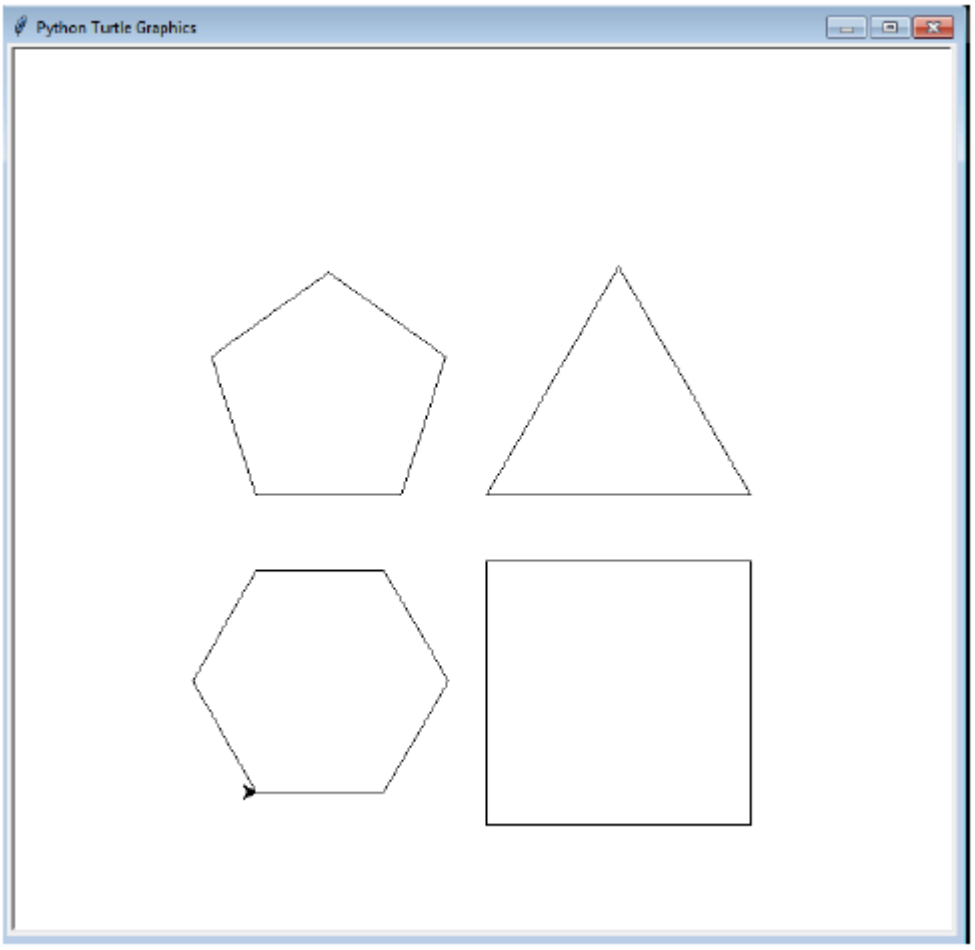
carré(200)
```

- Analysez le rôle de chaque ligne de code
- Complétez le programme précédent en insérant entre la première et la deuxième ligne, le code suivant :

```Python
t.up()
t.goto(0,-350)
t.down()
```

- Commentez chacune de ces lignes en expliquant leur rôle.

- Compléter encore le code afin d'obtenir le motif suivant :



💻 Exercice 4 - Bibliothèque `matplotlib`

Ce module permet de tracer des graphes.

- Après avoir créé un nouveau fichier, saisissez et exécutez le script suivant :

```python
import matplotlib.pyplot as plt
X = [1, 3, 4, 6]
Y = [2, 3, 5, 1]
plt.figure() # Initialisation de la figure
plt.plot(X, Y) # On trace la figure
plt.show() # On affiche la figure
```

- Lors d'un TP de physique on a obtenu le relevé de température suivant :.

Temp (°C)	**21**	**25**	**30**	**45**	**55**	**65**	**70**	**80**	**85**
Temps (s)	0	30	60	90	120	150	180	210	240

- Proposez un programme permettant de tracer la courbe caractéristique de la température en fonction du temps.

- Utilisez les instructions `plt.ylabel("Temp(°C) ")` et `plt.xlabel("Temps(s)")` afin d'ajouter des légendes aux axes.

💻 Exercice 4 - Bibliothèque `time`

- Importez le module `time` .
- Lancez plusieurs fois la fonction `time` du module, que constatez vous ?
- On va maintenant utiliser la fonction pour mesurer le temps d'exécution d 'un programme, pour cela, créez un nouveau fichier et :
- Importez le module `time`
- Lancez la fonction `time()` du module et stockez son retour dans une première variable nommée `start_time`
- Exécutez le code à mesurer, par exemple :

```Python
for k in range(1000000):
    pass
```

- Lancez à nouveau la fonction `time()` et stockez son retour dans une nouvelle variable nommée `end_time`
- Calculez la durée d'exécution en effectuant la différence entre les deux variables `start_time` et `end_time`
- Affichez la durée calculée
- La durée est en secondes, pourplus de lisibilité, affichez la durée en millisecondes.